Algorithms analysis and Design

BY : DR. BASEM ALIJLA

Chapter 2
Fundamentals of the Analysis of Algorithm Efficiency

OUTLINE

- **The Analysis Framework**
- **Asymptotic Notations & Basic Efficiency Classes**
- **Analysis of Non-recursive Algorithms**
- **Analysis of Recursive Algorithms**

The Analysis Framework

- **Analysis of algorithms:** an investigation of an algorithm's efficiency with respect to two resources:
  1. **running time** (Time efficiency or complexity)
  2. **memory space** (space efficiency or complexity)

  Efficiency can be studied in precise quantitative terms unlike **simplicity** ang **generality**

The Analysis Framework

- Approaches:
  1. Empirical analysis
  2. Theoretical analysis

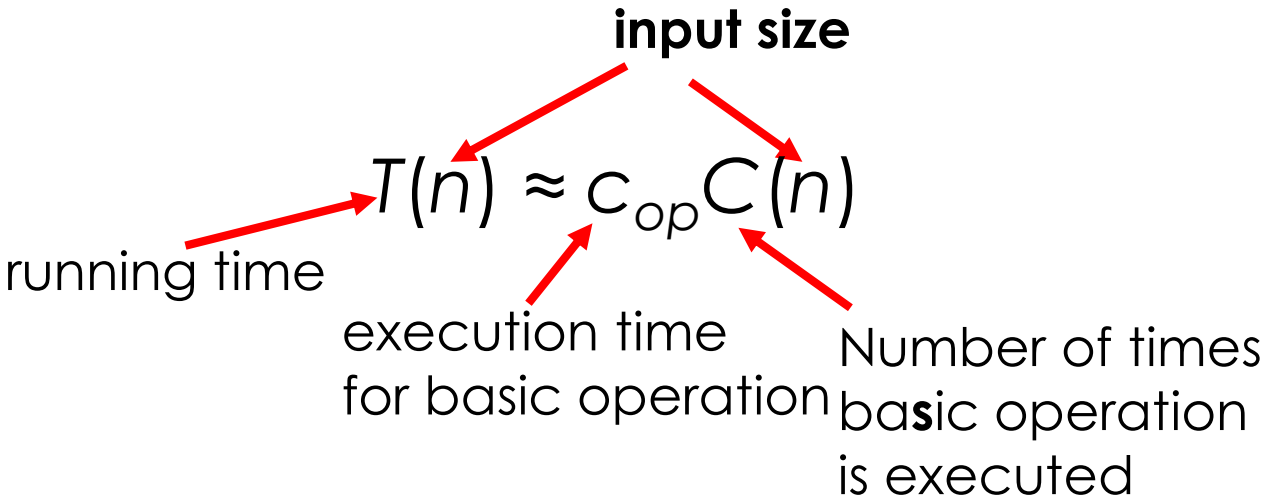
Empirical analysis of time efficiency

- Select a specific (typical) sample of inputs.
- Use physical unit of time (e.g., **milliseconds**) or
  Count actual number of basic **operation's executions**
- Analyze the empirical data

Theoretical analysis of time efficiency

- investigate an algorithm's efficiency as a function of some parameter ***n*** indicating the **algorithm's input size.**
- Time efficiency is analyzed by determining the number of repetitions of the ***basic operation*** as a function of ***input size***
- **Basic operations** are the most time consuming

Theoretical analysis of time efficiency



Theoretical analysis of time efficiency

- How much faster would this algorithm run on a machine that is 10 times faster than the one we have. **(10 Times)**
- how much longer will the algorithm run if we double its input size.

$$\text{Let } C(n) = \frac{1}{2}n\,(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$$

$$\frac{T(2n)}{T(n)} \approx \frac{C_{op}C(2n)}{C_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} \approx 4$$

Order of growth

- Order of Growth is the most important

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

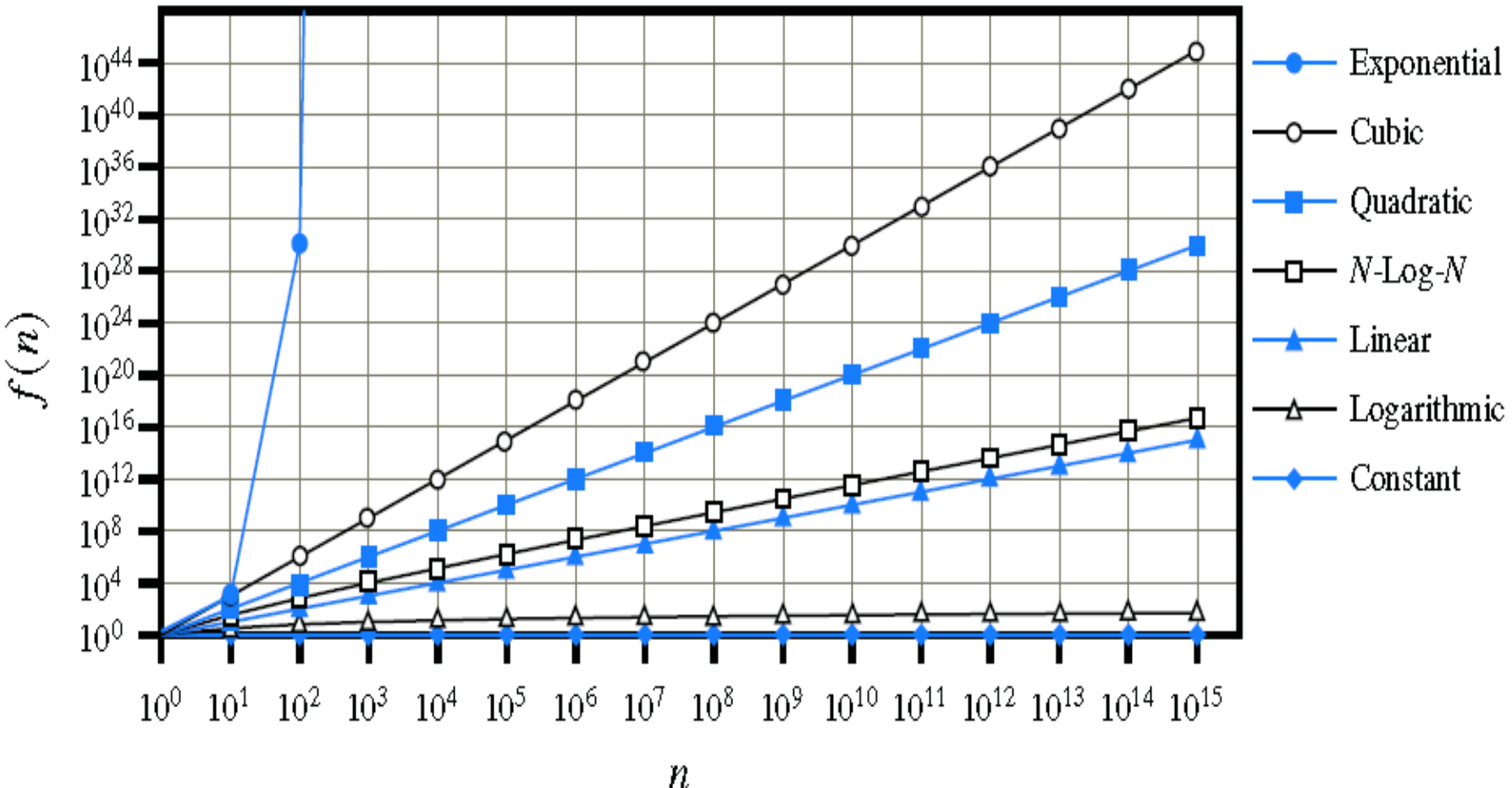
Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Order of growth

if runtime is...	time for n + 1	time for 2 n	time for 4 n
c lg n	c lg (n + 1)	c (lg n + 1)	c(lg n + 2)
c n	c (n + 1)	2c n	4c n
c n lg n	~ c n lg n + c n	2c n lg n + 2cn	4c n lg n + 4cn
c n^2	~ c n^2 + 2c n	**4c n^2**	16c n^2
c n^3	~ c n^3 + 3c n^2	8c n^3	64c n^3
c 2^n	c 2^{n+1}	c 2^{2n}	c 2^{4n}

runtime quadruples when problem size doubles

Seven Important Functions



Best-case, average-case, worst-case

For some algorithms efficiency depends on form of input:

- **Worst case:** $C_{worst}(n)$ – maximum over inputs of size n
- **Best case:** $C_{best}(n)$ – minimum over inputs of size n
- **Average case:** $C_{avg}(n)$ – "average" over inputs of size n

Best-case, average-case, worst-case

ALGORITHM *SequentialSearch*$(A[0..n-1], K)$

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1] and a search key K
//Output: The index of the first element of A that matches K
//              or −1 if there are no matching elements
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
if i < n return i
else return −1
```

Best-case, average-case, worst-case

- Find (3) is the Best Case

- Find (8) is the Average Case

- Find (12) is the Worst Case

Best-case, average-case, worst-case

Why Worst case is the most important :

Best-case is not representative.

Average-case analysis:

is ideal, but difficult to perform, because it is hard to determine the relative probabilities and distributions of various input instances for many problems.

Worst-case is not representative

but worst-case analysis is very useful. You can show that the algorithm will never be slower than the worst-case.

OUTLINE

- **The Analysis Framework**
- **Asymptotic Notations & Basic Efficiency Classes**
- **Analysis of Non-recursive Algorithms**
- **Analysis of Recursive Algorithms**

Asymptotic order of growth

- **O(*g*(*n*)):** class of functions *f*(*n*) that grow no faster than *g*(*n*). **(i.e., worst case)**
- **Θ(*g*(*n*)):** class of functions *f*(*n*) that grow at same rate as *g*(*n*) **(i.e., average or ideal case)**
- **Ω(*g*(*n*)):** class of functions *f*(*n*) that grow at least as fast as *g*(*n*) **(i.e., Best case)**

Big-oh : O(g(n))

- The big-Oh notation gives an **upper bound** on the growth rate of a function
- "$f(n)$ is $O(g(n))$"

  f(n) ≤ c(g(n)) for $n \geq n_0$
- Example: $2n + 10$ is $O(n)$

  $2n + 10 \leq cn$

  $(c - 2)\, n \geq 10$

  $n \geq 10/(c - 2)$

  Pick $c = 3$ and $n_0 = 10$

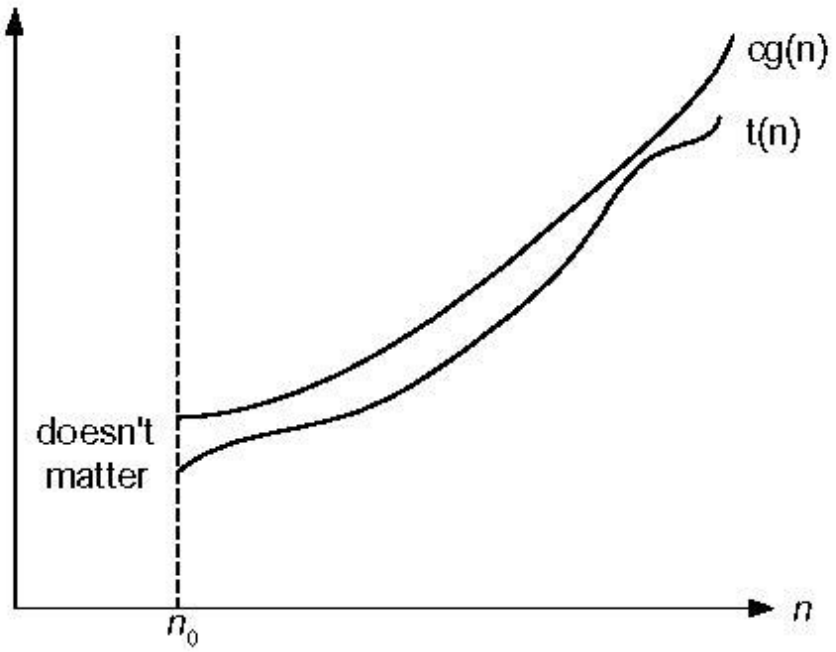


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

Big-theta : **Θ(*g*(*n*))**

The big-theta notation gives an **tide bound** on the growth rate of a function

f(n) is Θ(g(n)) if there are constants c' > 0 and c'' > 0 and an integer constant $n_0 \geq 1$ such that

$c'g(n) \leq f(n) \leq c''g(n)$ for $n \geq n_0$

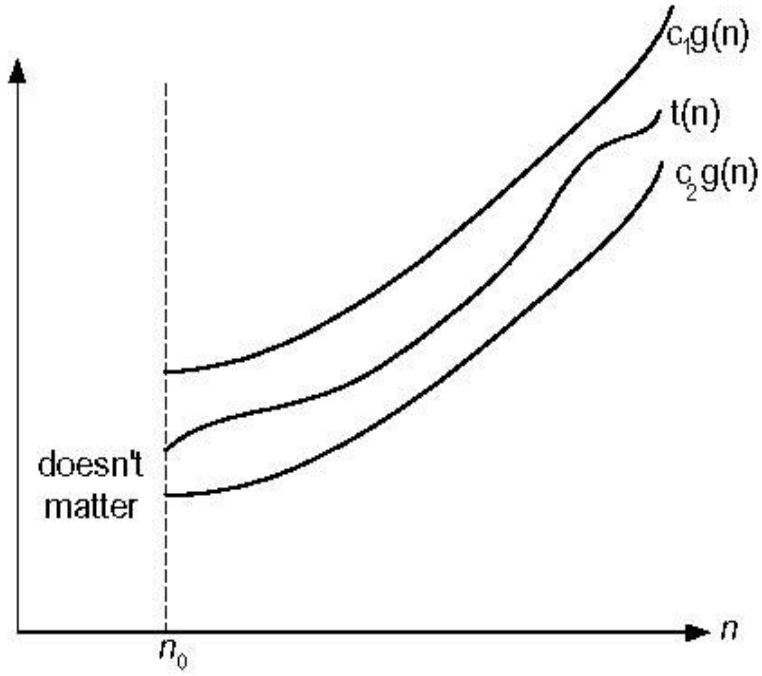


Figure 2.3 Big-theta notation: $t(n) \in \Theta(g(n))$

Big-omega: Ω(g(n))

The big-omega notation gives an **lower bound** on the growth rate of a function

f(n) is Ω(g(n)) if there is a constant c > 0, and an integer constant $n_0 \geq 1$ such that

$f(n) \geq c\ g(n)$ for $n \geq n_0$

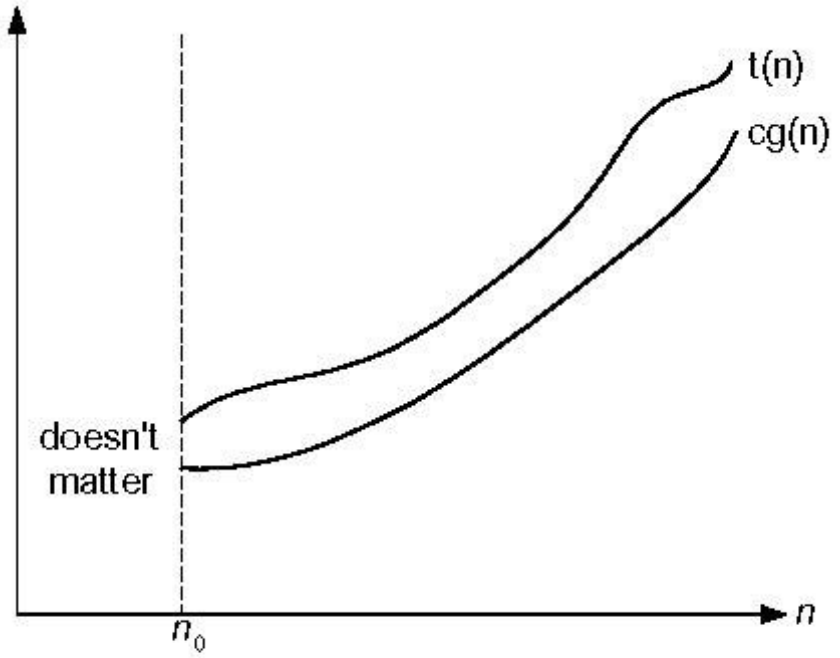


Fig. 2.2 Big-omega notation: $t(n) \in \Omega(g(n))$

OUTLINE

- **The Analysis Framework**
- **Asymptotic Notations & Basic Efficiency Classes**
- **Analysis of Non-recursive Algorithms**
- **Analysis of Recursive Algorithms**